

Software Supply Chain Attacks and Build Provenance

NG, Chun Sing (r14922186) OU, Chia-Yun (r14922072)

June 4, 2026

Abstract

Modern software development depends on package managers, CI/CD pipelines, and third-party registries. When developers run commands such as `npm install`, `pip install`, `mvn install`, or `conda install`, they implicitly trust a chain of transitive dependencies, maintainer accounts, build workflows, and release pipelines — not just the code they requested.

This report examines software supply chain attacks and the role of build provenance. It analyzes two npm ecosystem incidents: Axios, where stolen maintainer credentials and a `postinstall` script delivered a Remote Access Trojan through a normal installation; and TanStack, where malicious packages carried valid build provenance because the attacker compromised a shared CI/CD cache before the legitimate release workflow ran.

The central argument is that build provenance is necessary but not sufficient. It improves artifact traceability and supports install-time verification, but it cannot prove an artifact is benign, nor guarantee the build environment was clean. Effective defense requires provenance as one layer within a broader strategy.

Keywords: software supply chain security, build provenance, SLSA, Sigstore, npm, CI/CD security, dependency attack

1 Introduction

Modern development’s reliance on package managers creates a broad attack surface. A single install command may pull dozens of transitive packages and execute lifecycle scripts such as `preinstall` or `postinstall` with-

out manual review. An attacker therefore need not compromise the final application directly — instead, they target any trusted element in the chain: a dependency package, maintainer account, registry, build cache, CI/CD workflow, or release pipeline. A single upstream compromise can propagate malicious code to downstream projects through normal installation.

Build provenance records where a software artifact came from — the source commit, builder identity, and build workflow — and supports install-time verification of that chain. This report examines its limits through two npm incidents: the Axios compromise [12, 13, 14] illustrates what provenance can help detect, namely unauthorized publication through stolen maintainer credentials; the TanStack compromise [15, 16] illustrates what it cannot, as malicious packages carried *valid* provenance because the attacker had already subverted the build environment upstream. Together, the two cases support the central argument: **build provenance is necessary, but not sufficient**.

The remainder of this report is organized as follows. Section 2 provides background on supply chain attack surfaces, common attack forms, and the scale of the threat. Section 3 introduces build provenance: its concept, what it can and cannot prove, how verification works in practice, and the current state of ecosystem adoption. Section 4 presents the two case studies — Axios, where provenance would have been a concrete defensive control, and TanStack, where valid provenance accompanied a malicious release because the build environment had already been subverted. Section 5 synthesizes a layered defense strategy drawn from both incidents, Section 6 presents an experimental validation at the dependency layer, and Section 7 concludes.

2 Background

A software supply chain comprises every step that creates, transforms, and delivers a software artifact — source code, dependencies, build tools, CI/CD pipelines, package registries, and deployment mechanisms alike. A supply chain attack occurs when an attacker compromises any trusted node in this chain, abusing existing trust relationships so that malicious code enters downstream projects through normal development workflows.

Supply chain attacks span four layers [5]. At the **dependency layer**, attackers exploit malicious packages, typosquatting, dependency confusion, or lifecycle scripts. At the **source layer**, they target maintainer accounts,

commits, or release permissions. At the **build layer**, they abuse CI/CD workflows, cache poisoning, or OIDC tokens. At the **release layer**, they compromise registries or substitute artifacts. Prior work has proposed broader taxonomies of such attacks [28]; Table 1 summarizes the six forms that exploit these surfaces, covering the range from naming tricks to full pipeline compromise.

Table 1: Representative software supply chain attack forms

Attack Form	Description
Typosquatting	Publishes a package whose name closely resembles a popular one to mislead developers into installing the wrong package.
Dependency Confusion	Publishes a malicious package under the same name as a private package, causing the package manager to resolve the public (attacker-controlled) version.
Account Takeover	Steals a maintainer’s credentials to publish malicious versions through legitimate publishing channels.
Package Takeover	Acquires control of an abandoned or weakly governed package and releases a malicious version.
Code Injection	Inserts malicious code into a trusted package or artifact so it executes as part of normal workflows.
CI/CD Compromise	Abuses build workflows, caches, or tokens to make a legitimate pipeline produce a malicious artifact.

Supply chain failures now rank among the most significant application security risks. OWASP Top 10:2025 places them at A03 [3] — a shift from traditional defects such as injection or authentication failures toward external trust relationships: how an application is built, which components it depends on, who publishes them, and whether the final artifact can be verified. OWASP reports a 5.72% average incidence rate and 215,248 occurrences for A03:2025 [3, 4], though real-world frequency is likely higher: malicious packages and credential abuse often precede CVE database entries, so current scanning data underrepresents actual supply chain risk.

Four structural factors explain why this attack surface has grown so large. First, modern applications contain deep dependency graphs where a small number of direct dependencies can introduce many transitive ones. Second, package managers are highly automated: they resolve versions, download packages, and sometimes execute installation scripts without human review. Third, CI/CD systems have become central to build and release processes, making workflow permissions, runner environments, OIDC tokens, and build caches important attack surfaces. Fourth, developers rarely inspect every package version, tarball, or artifact hash manually — creating blind spots an attacker can exploit by compromising a single trusted node.

The scale and trajectory of the threat reinforce why structural defenses are needed. IBM 2025 breach-cost data places supply chain compromise at USD 4.91M average per attack — the second-costliest breach vector — with costs spanning incident response, credential rotation, and CI/CD rebuild [26]. Meanwhile, ReversingLabs reports malicious package detections increased by 73% in 2025, with npm accounting for nearly 90% of detected open-source malware [27], reflecting attackers' view of package ecosystems as a high-return surface given their scale, fast release rhythm, and highly transitive dependency graphs.

3 Build Provenance

3.1 Motivation

Common dependency defenses offer valuable but incomplete protection. Tools such as `npm audit` detect known CVEs, but a newly published malicious version may not yet appear in any vulnerability database. `npm ci` and lockfiles verify pinned versions and registry hashes, improving reproducibility — yet if a lockfile is generated during an attack window, it may preserve the compromised version. Dependabot automatically updates known-vulnerable dependencies, but still depends on vulnerability feeds and an automated upgrade may itself introduce a newly compromised version.

These limitations point to a deeper gap: traditional package installation workflows provide no verifiable, tamper-evident chain between source code and the installed artifact. When developers install a package, they usually cannot answer whether the artifact was built from the expected source commit, whether the build steps were unmodified, whether the registry tarball matches the CI output, or whether it was produced by the expected

workflow and builder identity. When these questions cannot be answered, attacks can remain silent — the package name, version, and registry page may all appear trustworthy while the artifact itself has been tampered with. Build provenance directly addresses this gap by creating a signed, verifiable, traceable evidence chain between source and installation [6].

3.2 Conceptual Model

Build provenance is metadata recording how a software artifact was produced: the source repository, commit hash, builder identity, build workflow, and artifact hash — summarized as *Built from commit X, by pipeline Y, at time T*. SLSA provides the framework defining what provenance should prove and at what assurance level [6]; Sigstore provides the signing infrastructure, using OIDC-bound keyless signing and the Rekor append-only transparency log [8]. When this record is signed and published to that log, a consumer can verify at install time that an artifact matches expectations.

Provenance can answer three questions that traditional installation cannot: whether the artifact came from the expected source commit (via a signed commit hash), whether the build ran on the expected pipeline (via OIDC-bound builder identity), and whether the registry tarball matches CI output (via an artifact hash) [6]. The core contribution is traceability — consumers can require machine-verifiable evidence about origin and build process rather than trusting package name and version alone.

Provenance is a passive record until verified. At install time, a consumer confirms the provenance signature has not been tampered with, compares the source repository, builder identity, and build type against expected values, and in higher-assurance settings recursively verifies dependencies [7]. Expected values can be declared by the maintainer, enforced by organizational policy, or inferred from prior attestations. This allows provenance to act as an install-time gate: a tool can warn when a version unexpectedly lacks an attestation, block installation when provenance disappears after a previously attested release, or verify and allow when a valid attestation is present [9].

3.3 Technology Stack and Ecosystem

Four layers work together in practice. SLSA defines the framework and build integrity levels [6]. Sigstore provides keyless signing and the Rekor transparency log [8]. Registries and CI platforms (npm, PyPI, GitHub Ac-

tions) generate and publish attestations on the producer side. Consumer-side tools such as npq [9] or Socket Firewall [10] act as pre-install gates, verifying provenance and evaluating package risk before installation.

Provenance support is maturing across major ecosystems: npm’s provenance became generally available in October 2023 (3,800 packages adopted during beta) [19, 20]; GitHub Artifact Attestations in June 2024 [18]; PyPI in November 2024 (17% of uploads attested by end of 2025) [21, 22]; with Maven, RubyGems, and Conda in various stages of opt-in or beta [23, 24, 25]. Adoption remains partial — the binding constraint is not tooling availability but maintainer awareness, migration overhead, and whether consumers actually verify attestations at install time rather than assuming safety from their mere existence.

4 Case Studies

4.1 Axios

Axios is a widely used JavaScript HTTP client with over 100 million weekly downloads [11]. On March 31, 2026, an attacker used stolen maintainer credentials to publish malicious versions during an exposure window of approximately three hours. Each malicious version injected an attacker-controlled hidden dependency whose `postinstall` script silently installed a Remote Access Trojan — meaning compromise occurred at installation time, not at runtime, before any user had run the application [12, 13].

The attack’s reach extended beyond individual developer machines. The incident affected OpenAI’s macOS app-signing pipeline, placing code-signing certificates for ChatGPT Desktop and Codex at risk [14]. A supply chain compromise in a development environment with build credentials or signing privileges can propagate further than an end-user compromise, since the affected artifacts may themselves be distributed downstream.

Critically, Axios had already adopted npm provenance: legitimate versions carried valid build attestations, but the malicious versions did not [12, 13]. A consumer-side tool performing provenance verification would have flagged the regression — a package that previously shipped attestations suddenly publishing versions without them is a strong indicator of unauthorized publication. This makes the Axios incident a case where build provenance, actively verified, would have been a concrete defense.

4.2 TanStack

On May 11, 2026, TanStack suffered an npm supply chain compromise in which 42 packages and 84 malicious versions were published within minutes, with payloads targeting cloud credentials, API tokens, and SSH keys. The incident was detected by an external researcher within 26 minutes [15]. Unlike the Axios compromise, the malicious versions carried *valid* build provenance — provenance correctly recorded that the packages were produced by TanStack’s legitimate workflow, because the attacker had subverted that workflow from within.

The attacker gained this access through cache poisoning. A fork pull request was used to poison a shared CI cache; when a later legitimate release workflow ran and restored that cache, it extracted an OIDC token from runner memory and used it to publish malicious packages through the legitimate release pipeline, generating valid provenance in the process [15]. The root cause is a misplaced trust boundary: fork PR workflows operate in a lower-trust context, but when they share caches with high-trust release workflows, a low-trust entry point can contaminate the entire process.

This demonstrates the core limitation of provenance: **provenance proves origin, not innocence** [16]. Provenance confirms that an artifact came from a specific repository, workflow, and builder — but it cannot prove the workflow was clean when it ran. If the build environment is compromised before the attestation is generated, provenance faithfully records a contaminated process. It therefore cannot substitute for CI/CD isolation, cache hardening, OIDC token scoping, and workflow permission auditing.

OpenAI’s response illustrates what layered defense looks like in practice: two employee devices were affected while controls were still in phased rollout. Post-Axios controls already deployed — hardening credential materials in CI/CD pipelines, configuring `minimumReleaseAge` to reject newly published versions, and validating provenance of new packages — show that provenance verification must be paired with install-time friction and credential protection to be effective [17].

The two incidents together define the boundary of provenance as a control. In the Axios case, provenance was decisive: the attacker could not replicate legitimate attestations, and a consumer verifying provenance would have detected the anomaly before installation. In the TanStack case, provenance was present and valid yet insufficient: because the attacker compromised the build environment before attestation was gener-

ated, provenance faithfully recorded a contaminated process. Together, they support the central argument — provenance is an effective control when the build environment is trustworthy, and a partial one when it is not.

5 Toward a Layered Defense Strategy

The two case studies show that no single defense mechanism can cover the entire supply chain attack surface: stolen credentials, lifecycle script abuse, CI/CD cache poisoning, and OIDC token theft each require different controls. Practical defense must therefore be layered.

5.1 Dependency Layer

The goal of the dependency layer is to reduce the probability that malicious or vulnerable packages enter a project. In practice, organizations should use lockfiles to control versions and hashes, and they should avoid unnecessary dependency updates. SBOMs can record dependency composition, while vulnerability scanners such as `npm audit` and OSV [32] can detect known vulnerabilities. However, these tools are insufficient for newly published malicious packages. Therefore, install-time gates such as `npq` [9] or Socket Firewall [10] should also be used to check package metadata, provenance status, and suspicious behavior. Static behavior analysis tools such as GuardDog [33] can further scan package source code for suspicious patterns, providing a detection layer that does not depend on CVE records. In high-risk environments, a minimum release age policy can reduce the probability that a newly published malicious version enters a project immediately [17].

5.2 Source Layer

The source layer protects source repositories and maintainer accounts. If a maintainer account is stolen, the attacker may directly publish malicious versions or modify the release process. Therefore, multi-factor authentication, protected branches, mandatory code review, and restricted publishing permissions are necessary controls. Organizations should also monitor abnormal commits, releases, token usage, and maintainer activity so that suspicious behavior from trusted accounts can be detected early.

5.3 Build Layer

The TanStack incident shows that build layer isolation is critical. Organizations must strictly separate untrusted pull request workflows from trusted release workflows, and should avoid sharing high-trust caches between fork pull request workflows and release workflows. GitHub Actions tokens, OIDC tokens, and registry tokens should follow the principle of least privilege, and credential materials should not appear in runner memory, logs, or locations readable by low-trust processes. Cache restore paths, build scripts, and release scripts should be treated as part of the security boundary, not merely as performance optimization tools [15, 16].

5.4 Release Layer

The goal of the release layer is to make artifacts verifiable during publication and installation. Organizations should use signed attestations and build provenance to record source commits, builder identities, artifact hashes, and build workflows in verifiable metadata [6]. More importantly, consumers should verify provenance at install time rather than merely generating provenance at publish time [7]. If a package previously had an attestation but a new version suddenly lacks one, installation tools should at least warn the user, and may block installation according to policy [9].

6 Experimental Validation

To ground the theoretical analysis in an observable demonstration, we conducted a controlled experiment at the dependency layer. The central question: if a malicious npm package has no CVE and appears in no vulnerability database, but executes clearly suspicious behavior at install time, can common defensive tools detect it?

We built a simulated malicious package, `demo-malicious-tool` [29, 30], modelled on the Axios-style install-time attack. Its `postinstall` script uses `INIT_CWD` to locate the installing project, reads its `.env`, serializes `process.env`, and sends a JSON payload via HTTP POST to a local test listener. A demo project depending on this package was then scanned with four tools spanning three defense categories: `npm audit` and OSV [32] (known-vulnerability matching), `npq` [9] (supply chain metadata signals), and `GuardDog` [33] (static behavior analysis). A real package, `is-number@7.0.0` [31] — feature-stable since 2018 and without provenance

— was included as a benign counterpart to observe false-positive behavior.

Table 2: Expected and actual scan results for `demo-malicious-tool`

Tool	Category	Expected	Actual
npm audit	CVE / advisory lookup	Not flagged	Not flagged
OSV	Vulnerability database	Not flagged	Not flagged
npq	Metadata signals	Inconclusive	Not found (GitHub dependency)
GuardDog	Static behavior analysis	Flagged	Flagged (3 rules)

The results match expectations. `npm audit` and `OSV` did not flag the package because no advisory record exists for a newly created package — this is a design boundary, not a failure: these tools answer whether a known vulnerability has been recorded, not whether the code behaves maliciously. `npq` could not evaluate the package because it is a GitHub dependency rather than a registry package, exposing a coverage gap for non-registry sources. `GuardDog` flagged three behavioral rules — `npm-serialize-environment`, `npm-exfiltrate-sensitive-data`, and `npm-install-script` — detecting the attack pattern from source code without any prior record.

`is-number@7.0.0` was flagged by both `npq` (expired domain, missing provenance, package age) and `GuardDog` (one rule). As a widely used and benign package, these are false positives, confirming that metadata signals and behavioral rules require human interpretation rather than automated rejection.

One important caveat applies to the experiment as a whole: `demo-malicious-tool` is a GitHub dependency with no registry presence or download history. In practice, developers would have reason to question installing such an unknown package; real supply chain attacks are more subtle, exploiting compromised existing packages or naming proximity rather than unknown repositories. The experiment therefore demonstrates the detection gap under controlled conditions. Its finding — that behavioral analysis can detect malicious install-time patterns independently of vulnerability databases — should be read as a proof of concept, not a simulation

of a complete real-world attack.

7 Conclusion

Software supply chain attacks succeed because they exploit trust across every layer of modern development: package managers, maintainer accounts, registries, build workflows, and release pipelines. Attackers need not breach an application directly — a single compromised upstream node propagates malicious code through normal installation.

The two case studies and the experimental validation together define the boundary of provenance as a control. Axios shows that a provenance regression check would have been decisive against a stolen-credential attack. TanStack shows that provenance is insufficient when the build environment is subverted from within. The experiment further shows that CVE-based scanning misses newly published malicious packages, while behavioral analysis can detect suspicious patterns without any prior record. Across all three cases, no single mechanism covers the full attack surface.

Build provenance is therefore necessary but not sufficient. Effective defense requires it as one layer among many — behavioral analysis, dependency scanning, lockfile discipline, minimum release age policies, maintainer account protection, CI/CD isolation, and credential restriction — each answering a different question, together reducing a systemic risk that no individual control can eliminate alone.

References

- [1] National Institute of Standards and Technology (2024). *NIST SP 800-204D: Strategies for the Integration of Software Supply Chain Security in DevSecOps CI/CD Pipelines*. <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-204D.pdf>
- [2] OWASP. *Software Supply Chain Security Cheat Sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Software_Supply_Chain_Security_Cheat_Sheet.html Accessed: June 3, 2026.
- [3] OWASP (2025). *A03:2025 Software Supply Chain Failures*. https://owasp.org/Top10/2025/A03_2025-Software_Supply_Chain_Failures/
- [4] OWASP (2025). *OWASP Top 10:2025 Introduction and Methodology*. https://owasp.org/Top10/2025/0x00_2025-Introduction/
- [5] SLSA (2023). *Supply Chain Threats*. <https://slsa.dev/spec/v1.0/threats-overview> Accessed: June 3, 2026.
- [6] SLSA (2023). *About SLSA*. <https://slsa.dev/spec/v1.2/about> Accessed: June 3, 2026.
- [7] SLSA (2023). *Verifying Artifacts*. <https://slsa.dev/spec/v1.0/verifying-artifacts> Accessed: June 3, 2026.
- [8] OpenSSF. *Sigstore*. <https://www.sigstore.dev/> Accessed: June 3, 2026.
- [9] Liran Tal. *npq: Safely install packages with npm or yarn by auditing them as part of your install process*. <https://github.com/lirantal/npq> Accessed: June 3, 2026.
- [10] Socket. *Introducing Socket Firewall*. <https://socket.dev/blog/introducing-socket-firewall> Accessed: June 3, 2026.
- [11] npm. *axios: Weekly Downloads*. <https://www.npmjs.com/package/axios> Accessed: June 3, 2026.

- [12] Axios (2026). *Post Mortem: axios npm supply chain compromise*.
<https://github.com/axios/axios/issues/10636>
- [13] Microsoft Security Blog (2026). *Mitigating the Axios npm supply chain compromise*.
<https://www.microsoft.com/en-us/security/blog/2026/04/01/mitigating-the-axios-npm-supply-chain-compromise/>
- [14] OpenAI (2026). *Our response to the Axios developer tool compromise*.
<https://openai.com/index/axios-developer-tool-compromise>
- [15] TanStack (2026). *Postmortem: TanStack npm supply-chain compromise*. <https://tanstack.com/blog/npm-supply-chain-compromise-postmortem>
- [16] TanStack (2026). *Hardening TanStack After the npm Compromise*.
<https://tanstack.com/blog/incident-followup>
- [17] OpenAI (2026). *Our response to the TanStack npm supply chain attack*. <https://openai.com/index/our-response-to-the-tanstack-npm-supply-chain-attack>
- [18] GitHub Blog (2024). *Artifact Attestations is generally available*.
<https://github.blog/changelog/2024-06-25-artifact-attestations-is-generally-available/>
- [19] GitHub Blog (2023). *Introducing npm package provenance*.
<https://github.blog/security/supply-chain-security/introducing-npm-package-provenance/>
- [20] Sigstore Blog (2023). *npm's Sigstore-powered provenance goes GA*.
<https://blog.sigstore.dev/npm-provenance-ga/>
- [21] PyPI Blog (2024). *PyPI now supports digital attestations*.
<https://blog.pypi.org/posts/2024-11-14-pypi-now-supports-digital-attestations/>
- [22] PyPI Blog (2025). *PyPI 2025 Year in Review*. <https://blog.pypi.org/posts/2025-12-31-pypi-2025-in-review/>
- [23] Sonatype Central (2025). *Sigstore Signature Validation via Portal*.
https://central.sonatype.org/news/20250128_sigstore_signature_validation_via_portal/

- [24] RubyGems Blog (2025). *February 2025 Updates*. <https://blog.rubygems.org/2025/03/19/february-rubygems-updates.html>
- [25] prefix.dev (2025). *Securing the Conda package supply chain with Sigstore*. <https://prefix.dev/blog/securing-the-conda-package-supply-chain-with-sigstore>
- [26] IBM (2025). *Cost of a Data Breach Report 2025 / Attack vector overview*. <https://www.ibm.com/think/topics/attack-vector>
- [27] ReversingLabs (2026). *Software Supply Chain Security Report 2026*. <https://www.reversinglabs.com/resources/software-supply-chain-security-report-2026>
- [28] ScienceDirect (2025). *A taxonomy and analysis of software supply chain attacks*. <https://www.sciencedirect.com/science/article/pii/S2214212625003606>
- [29] OU, Chia-Yun (2026). *demo-malicious-tool: Simulated malicious npm package for supply chain experiment*. <https://github.com/0uChiaYun/demo-malicious-tool>
- [30] OU, Chia-Yun; NG, Chun Sing (2026). *NTU_SupplyChainAttacks: npm Supply Chain Risk Gate*. https://github.com/0uChiaYun/NTU_SupplyChainAttacks
- [31] Jon Schlinkert. *is-number (npm package)*. <https://www.npmjs.com/package/is-number> Accessed: June 3, 2026.
- [32] Google (2021). *OSV: Open Source Vulnerabilities*. <https://osv.dev/> Accessed: June 3, 2026.
- [33] Datadog (2022). *GuardDog: Identify malicious PyPI and npm packages*. <https://github.com/DataDog/guarddog> Accessed: June 3, 2026.